Plant Leaf Disease Classification Using a Custom Convolutional Neural Network

Amey Ravindra Bonde
Department of Electrical Engineering
COEP Technological University
Pune, Maharashtra, India
bondear24.elec@coeptech.ac.in

***Abstract*—Image-based plant disease classification is frequently evaluated on curated leaf-image datasets, yet high aggregate scores on controlled images do not necessarily establish robustness to field conditions. This paper audits and evaluates a custom convolutional neural network (CNN) implemented for 38 PlantVillage classes, with emphasis on reproducibility, class-count intervention, validation methodology, and the distinction between model confidence and diagnostic reliability. The inspected pipeline copies the color-image directory to a local runtime, caps three selected classes at 2,000 images each, resizes images to 224×224 RGB, scales pixel values to $[0, 1]$, and uses an 80:20 training-validation split. The Sequential CNN contains two convolutional blocks followed by a flattened dense classifier and has 47,805,158 trainable parameters. The notebook reports 35,492 training and 8,859 validation images. After six epochs, the selected checkpoint achieves 89.12% validation accuracy and 0.4948 validation loss; macro F1 is 0.8762 and weighted F1 is 0.8904. The 99.74% training accuracy recorded at the final epoch indicates a substantial generalization gap. Class-wise results are heterogeneous: Tomato Early blight recall is 0.5950 and Potato healthy recall is 0.6000, whereas Corn common rust recall is 0.9874. A Streamlit interface is also provided, but its displayed 100.00% softmax score for one example is not evidence of calibration or general accuracy. The main contribution is an evidence-grounded, reproducible baseline and a critical account of what the available experiment can and cannot establish. No independent test set or field evaluation is available, so the findings are limited to the reported validation split.**

***Index Terms*—plant disease classification, convolutional neural network, PlantVillage, class imbalance, image classification, model evaluation, generalization, reproducibility**

I. Introduction

Plant diseases can reduce crop quality and yield, while timely diagnosis may require access to agronomic expertise. Computer vision systems are attractive as decision-support tools because they can map leaf photographs to candidate crop–disease categories. Early work established the value of open, labelled plant-health image repositories for machine learning and crowdsourced diagnostics [1], [2]. The present project uses the color-image version of PlantVillage and a custom CNN to classify 38 leaf categories.

A central methodological issue is that benchmark performance and field performance are different claims. Mohanty *et al.* reported 99.35% accuracy on a held-out test set but only 31.4% accuracy on images gathered from different online sources, illustrating a substantial distribution shift [2]. This distinction is important for any prototype that may eventually be used on photographs captured with phones, under variable illumination, backgrounds, focus, cultivar, and disease stage. A model's output should therefore be interpreted as a prediction among its known labels, not as an expert diagnosis.

This paper does not claim a new CNN architecture or state-of-the-art performance. Instead, it asks: *What performance and limitations are supported by the executed notebook for a custom 38-class CNN after a targeted class-count reduction, and what does the evidence imply for reproducibility and deployment?* This question is answerable from the provided notebook outputs and source code. It also avoids overclaiming novelty from an application that combines standard convolution, pooling, dense layers, and softmax.

The study makes four evidence-bounded contributions:

1) It documents the actual dataset curation and split counts, including the limitation that only three high-count classes are capped.
2) It reconstructs the model and training configuration from the executed Keras notebook.
3) It reports aggregate and class-wise validation results and analyses the training-validation gap.
4) It audits the Streamlit inference path and clarifies the difference between a softmax score, validation accuracy, and real-world diagnostic reliability.

The remainder of the paper reviews relevant literature, describes the experimental pipeline, presents results, discusses validity threats, and outlines experiments required before making stronger claims.

II. Related Work

A. PlantVillage and image-based diagnosis

Hughes and Salathé introduced an open-access repository of plant-health images intended to enable mobile diagnostics and machine-learning research [1]. The PlantVillage benchmark is widely used because it contains labelled healthy and diseased leaf images across multiple crops. Its value is the ability to compare algorithms on a common image classification problem; however, the capture conditions are more controlled than many field settings.

Mohanty *et al.* trained deep CNNs on 54,306 images spanning 14 crop species and 26 diseases or healthy categories

[2]. Their 99.35% held-out accuracy is often cited as evidence of technical feasibility, but their external-image result of 31.4% is equally important: it shows that random in-dataset evaluation can overstate the readiness of a model for uncontrolled imagery. Thus, a comparison between their test result and the present 89.12% validation result is not a controlled algorithm comparison. The splits, model architecture, class treatment, and evaluation protocols differ.

B. CNN architecture and transfer learning

CNNs learn local spatial filters and compose them into increasingly abstract representations. A convolution layer applies shared kernels across an image, while pooling reduces spatial resolution. Dense layers combine extracted features for classification. This project follows a simple CNN-from-scratch design, rather than using a pretrained backbone. This is a reasonable baseline for learning and implementation, but it may require more data and training care than transfer learning. Studies comparing deep architectures for plant disease recognition have explored models such as VGG, Inception, and residual networks, often reporting strong scores on curated datasets [3], [4]. Those results should not be treated as direct benchmarks here because datasets, splits, preprocessing, and class sets vary.

C. Dataset shift, class imbalance, and evaluation

A systematic review of deep learning for plant disease tasks identifies dataset scarcity, class imbalance, image quality, and generalization as recurring challenges [5]. Class imbalance matters because aggregate accuracy can be dominated by common classes, while minority classes may have high variance or poor recall. Macro-averaged metrics assign equal weight to each class, whereas weighted metrics reflect class support. Both are useful, but neither replaces inspection of per-class results.

A separate issue is evaluation design. If related images of the same physical leaf or acquisition session are split across train and validation sets, the measured result may be optimistic. The supplied notebook uses Keras' directory-based 80:20 validation split; it does not document leaf-group metadata or an independent test set. This paper therefore reports the metric as validation accuracy and avoids describing it as test performance.

D. Research gap and positioning

The literature already establishes that CNNs can classify PlantVillage images and that out-of-distribution performance can be substantially worse than controlled-dataset accuracy. The defensible gap addressed here is narrower and practical: an auditable account of one implemented baseline, including a partial class-count intervention, the exact train-validation outputs, class-wise weaknesses, and app-level confidence presentation. The contribution is reproducibility and critical evaluation, not a novel algorithm. No superiority claim is made.

III. Research Questions

The paper addresses the following questions:

- **RQ1:** What validation performance does the implemented 38-class CNN achieve under the notebook's recorded data pipeline?
- **RQ2:** How heterogeneous are precision, recall, and F1 across classes, even when aggregate accuracy is reasonably high?
- **RQ3:** What evidence indicates overfitting or limitations in generalization?
- **RQ4:** Which implementation and evaluation limitations must be addressed before field-oriented claims are justified?

These questions are descriptive and diagnostic. Since the notebook contains no controlled model comparison, the paper does not attempt to identify a causal benefit from the class-reduction operation.

IV. Dataset and Experimental Setup

A. Dataset source and class structure

The notebook reads a PlantVillage `color` directory arranged in one folder per class. The executed exploratory-data-analysis cell reports 38 folders and 54,305 supported images before the reduction cell, with a minimum class count of 152 and maximum of 5,507. These counts are the notebook's own observed values and may differ slightly from other distributions of PlantVillage because mirrored releases, excluded files, and dataset versions can differ [6]. The paper does not assume that every published PlantVillage copy is byte-for-byte identical to the local data.

Figure 1 reproduces the class-frequency visualization generated by the notebook before reduction. The spread is substantial, motivating the project's attempt to reduce the dominance of selected high-count classes. The figure is not a post-reduction distribution and must be interpreted accordingly.

B. Targeted class reduction

The curation cell sets a target of 2,000 images and applies it only to Orange citrus greening, Tomato Yellow Leaf Curl Virus, and Soybean healthy. For each selected folder, the cell randomly samples excess file names and removes the corresponding files from the local dataset copy. Its printed output confirms 2,000 remaining images for each selected class. This action reduces those folders but does not equalize all 38 class counts. The other classes retain their original counts, including small classes.

The intervention is destructive within the copied runtime directory. A more reproducible procedure would retain the raw dataset, write a manifest of selected files, and create a derived subset without deleting the only local copy. The notebook sets Python, NumPy, and TensorFlow seeds to 42; nevertheless, exact reproducibility also depends on file ordering, the random-state progression, library versions, and execution order. The notebook does not preserve a complete before/after count table or removed-file manifest, which limits retrospective verification.

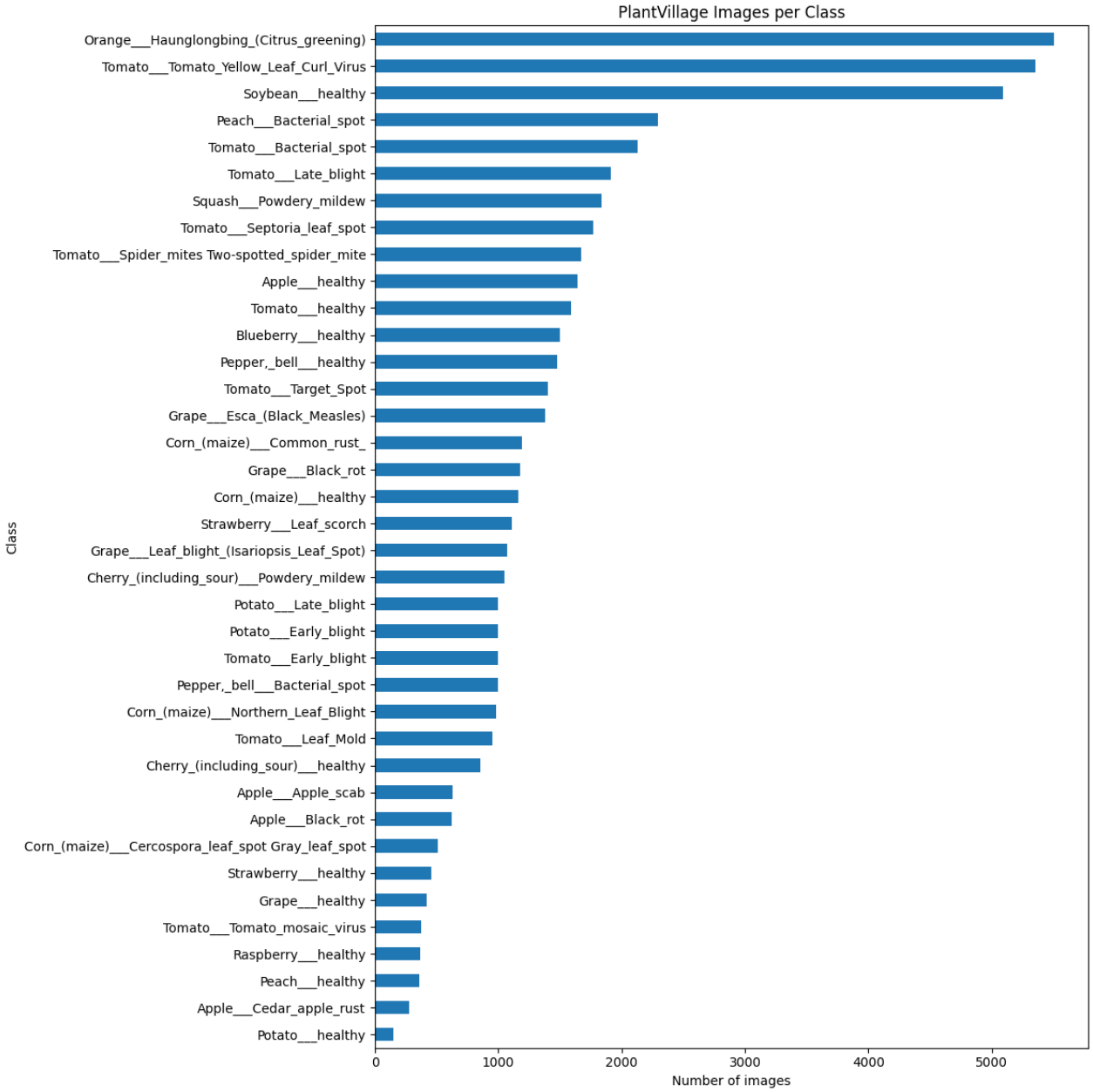


Fig. 1. Class-frequency plot generated by notebook cell 15 before the targeted reduction. The notebook reports 38 classes, 54,305 images, a minimum class count of 152, and a maximum of 5,507.

C. Train-validation split

The notebook configures `ImageDataGenerator` with `validation_split=0.20`, uses `subset=training` and `subset=validation`, and sets validation shuffling to false. It reports 35,492 training images and 8,859 validation images, for 44,351 total generator examples. Figure 2 summarizes this split and highlights a crucial boundary: no separate test set is created in the inspected notebook.

The initial EDA count (54,305) and later generator total (44,351) refer to different stages. The notebook explicitly removes excess images from three classes, but does not provide a per-class before/after audit trail sufficient to reconcile every file-level difference. The paper therefore reports both counts with their stage labels rather than asserting an exact causal decomposition of the difference.

D. Preprocessing

Images are resized to 224×224 pixels, represented as RGB, and scaled by dividing each pixel by 255. Let the input image be $I \in \{0, \ldots, 255\}^{H \times W \times 3}$. The model input is

$$\tilde{I}_{i,j,c} = \frac{I_{i,j,c}}{255}, \tag{1}$$

with $\tilde{I} \in [0,1]^{224 \times 224 \times 3}$. Labels are represented categorically using one-hot vectors. The inspected generator does not config-

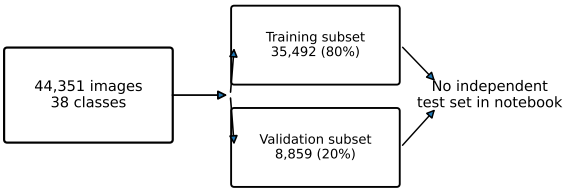


Fig. 2. Split methodology reconstructed from the executed generator configuration. The validation subset is used for model selection and reported evaluation; an independent test set is absent.

TABLE I
EXPERIMENTAL CONFIGURATION EXTRACTED FROM THE NOTEBOOK

Item	Recorded setting
Dataset	PlantVillage color directory; 38 classes
Pre-reduction count	54,305 images in EDA output
Training subset	35,492 images
Validation subset	8,859 images
Image dimensions	$224 \times 224 \times 3$
Batch size	32
Scaling	Pixel values divided by 255
Augmentation	None configured in generator
Optimizer	Adam, initial learning rate 10^{-3}
Loss	Categorical cross-entropy
Metric	Accuracy
Epochs	6
Checkpoint	Best validation accuracy
Scheduler	Reduce LR on validation-loss plateau; factor 0.3
Random seeds	Python, NumPy, TensorFlow = 42
Framework	TensorFlow/Keras 2.21.0
Hardware	GPU reported; exact model unspecified
Independent test set	Not present

ure geometric or photometric data augmentation. Consequently, this experiment should be described as training with rescaling and a validation split, not as an augmented training pipeline.

E. Experimental configuration

Table I lists the configuration supported by the notebook. TensorFlow reports version 2.21.0 and at least one GPU device, but the saved output does not identify the exact GPU model. The dependency file supplied with the app does not pin package versions. These details limit exact environment recreation.

V. CNN Architecture and Training

A. Architecture

The model is a Keras Sequential network with an input of shape $224 \times 224 \times 3$, a 3×3 convolution with 32 ReLU filters, 2×2 max pooling, a second 3×3 convolution with 64 ReLU filters, another 2×2 max pooling, a flatten operation, a 256-unit ReLU dense layer, and a 38-unit softmax output. The convolution layers use the default valid padding, consistent with the summary shapes of $222 \times 222 \times 32$ and $109 \times 109 \times 64$.

Figure 3 reconstructs the architecture from the model definition and summary. It is an original diagram of the supplied model, not a reproduced figure from external literature.

For a convolutional layer with kernel size $k_h \times k_w$, input channels C_{in}, and C_{out} filters, the number of trainable parameters is

$$P_{\text{conv}} = (k_h k_w C_{\text{in}} + 1) C_{\text{out}}, \tag{2}$$

where the additional parameter per filter is its bias. For the first convolution this yields $(3 \cdot 3 \cdot 3 + 1) \cdot 32 = 896$ parameters. The second convolution has $(3 \cdot 3 \cdot 32 + 1) \cdot 64 = 18,496$ parameters. After pooling, the feature tensor has shape $54 \times 54 \times 64$, which flattens to 186,624 values. Connecting these values to 256 hidden units produces

$$P_{\text{dense}} = (186624 + 1) \cdot 256 = 47,776,000. \tag{3}$$

This single dense layer accounts for nearly all model parameters and is a key architectural consideration. The total is 47,805,158 trainable parameters (approximately 182.36 MiB under the notebook's reported storage convention). A global-average-pooling head or smaller feature representation could reduce parameter count, but neither alternative was evaluated in this experiment.

B. Output and objective

For input image x, the network outputs class scores transformed by softmax:

$$p_k(x) = \frac{\exp(z_k(x))}{\sum_{j=1}^{38} \exp(z_j(x))}, \tag{4}$$

where z_k is the logit for class k. The predicted label is $\hat{y} = \arg\max_k p_k(x)$. The categorical cross-entropy objective for one-hot label vector y is

$$\mathcal{L}(y, p) = -\sum_{k=1}^{38} y_k \log(p_k). \tag{5}$$

The optimizer is Adam with initial learning rate 10^{-3}. A checkpoint callback saves the model with the best observed validation accuracy. A ReduceLROnPlateau callback monitors validation loss, with factor 0.3, patience 2, and minimum learning rate 10^{-6}. The best checkpoint is reloaded before the evaluation cell and final model export.

C. Training protocol and model selection

The notebook trains for six epochs. The validation subset serves both as the callback/model-selection signal and as the reported evaluation set. This is a common iterative development pattern but introduces selection optimism: the reported score is not an independent estimate after model selection. A final independent test set, untouched during checkpoint selection, is needed for a more defensible final generalization estimate. Since that set is absent, the present paper deliberately reports validation metrics only.

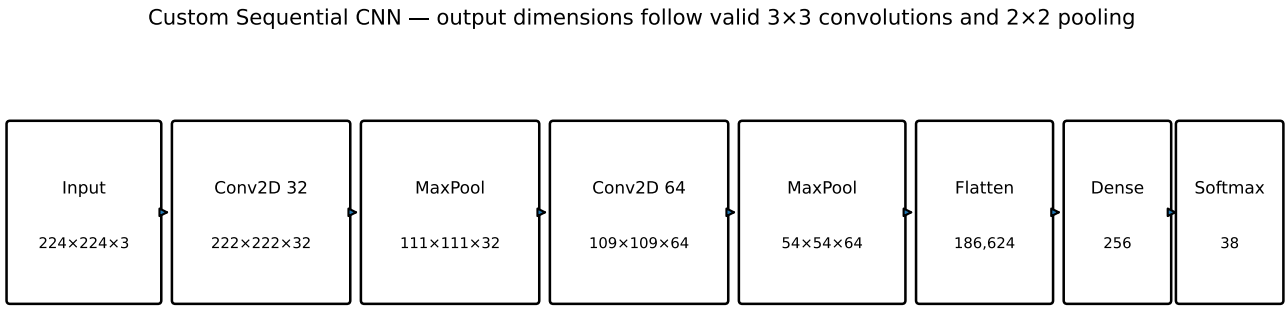


Fig. 3. CNN architecture reconstructed from notebook cells 29–30. Tensor shapes and parameter count follow the executed Keras model summary.

VI. Evaluation Metrics

Let TP_c, FP_c, and FN_c denote true positives, false positives, and false negatives for class c. Per-class precision, recall, and F1 are

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \tag{6}$$

$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c}, \tag{7}$$

$$F1_c = \frac{2\,\text{Precision}_c\,\text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}. \tag{8}$$

Macro averages take the unweighted mean across the 38 classes, while weighted averages weight each class by its support. Accuracy is the fraction of correct predictions over all validation examples. These metrics are complementary: macro F1 highlights class-level disparities, while weighted F1 is more influenced by classes with larger support.

VII. Results

A. Training dynamics

Table II reproduces the epoch-level values logged in the notebook. Training accuracy rises from 65.74% to 99.74%. Validation accuracy rises from 80.22% to 89.12% overall but dips during epochs 4 and 5. Validation loss falls through epoch 3, then rises to 0.6772 at epoch 5 before dropping to 0.4948 at epoch 6. The checkpoint selected at epoch 6 is therefore the best of the six recorded validation-accuracy checkpoints.

Figures 4 and 5 are the actual plots saved from notebook cells 38 and 39. The difference between training and validation accuracy at epoch 6 is approximately 10.62 percentage points. The training loss also approaches zero while validation loss remains non-zero and fluctuates. These patterns are consistent with overfitting or a mismatch between the training and validation distributions. They do not identify a unique cause; stochastic optimization, the selected checkpoint, class distribution, and lack of augmentation may all contribute.

TABLE II
Recorded Training History

Epoch	Train acc.	Val. acc.	Train loss	Val. loss
1	65.74%	80.22%	1.3046	0.6639
2	86.55%	81.25%	0.4310	0.6104
3	93.26%	85.25%	0.2114	0.5062
4	95.87%	85.22%	0.1247	0.5898
5	96.98%	84.41%	0.0957	0.6772
6	99.74%	89.12%	0.0110	0.4948

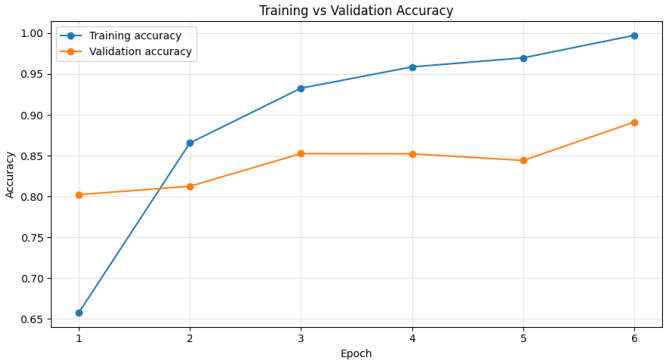


Fig. 4. Training and validation accuracy across the six recorded epochs, exported from notebook cell 38.

B. Aggregate validation performance

Table III gives the aggregate metrics from the classification report in notebook cell 40. The model reaches 89.12% accuracy and 0.4948 loss on 8,859 validation examples. Macro F1 is 0.8762, slightly lower than weighted F1 at 0.8904, indicating that the overall weighted result is somewhat more favorable than the equal-class average. This is consistent with heterogeneous per-class results, although the gap alone does not prove that class imbalance caused the weaker classes.



Fig. 5. Training and validation loss across the six recorded epochs, exported from notebook cell 39.

TABLE III
AGGREGATE VALIDATION METRICS REPORTED BY THE NOTEBOOK

Metric	Value
Validation accuracy	0.8912
Validation loss	0.4948
Macro precision	0.8802
Macro recall	0.8741
Macro F1-score	0.8762
Weighted precision	0.8909
Weighted recall	0.8912
Weighted F1-score	0.8904
Validation examples	8,859

C. Class-wise performance

Figure 6 plots the per-class precision, recall, and F1 values transcribed from the notebook's printed classification report. It is generated from those reported metrics, not estimated from a screenshot. The lowest recall among the listed classes is Tomato Early blight at 0.5950, followed by Potato healthy at 0.6000 and Corn gray leaf spot at 0.6961. Corn common rust has 0.9874 recall, Corn healthy 0.9914, and Orange citrus greening 0.9850. A model that performs strongly on some classes may still fail to detect a meaningful fraction of examples in another class.

Table IV lists several classes that illustrate the range of observed behavior. The table is not intended to cherry-pick a representative mean; it highlights the lowest recall values and a few high-recall reference classes to direct future error analysis.

The small support for Potato healthy ($n = 30$) means its recall estimate is less stable than that of classes with several hundred validation examples. One incorrect prediction changes its recall by approximately 3.33 percentage points. Tomato Early blight has larger support ($n = 200$) but low recall and precision, making it a higher-priority candidate for targeted error inspection. These values establish observed weaknesses but do not explain their causes. Potential explanations include visually overlapping symptoms, within-class variation, sample count, or dataset-specific image properties; these hypotheses require image-level review and controlled experiments.

TABLE IV
SELECTED CLASS-WISE VALIDATION RESULTS

Class	P	R	F1	n
Tomato Early blight	0.6919	0.5950	0.6398	200
Potato healthy	0.7826	0.6000	0.6792	30
Corn gray leaf spot	0.7717	0.6961	0.7320	102
Apple scab	0.7967	0.7778	0.7871	126
Tomato yellow leaf curl	0.9261	0.9400	0.9330	400
Corn common rust	0.9833	0.9874	0.9853	238
Corn healthy	0.9871	0.9914	0.9892	232
Orange citrus greening	0.9704	0.9850	0.9777	400

D. Confusion matrix

Figure 7 shows the confusion matrix generated from predictions on the validation generator. It is an actual notebook output, not a fabricated matrix. Because the matrix has 38 classes, the figure is large and should be read at high resolution. It can reveal which pairs of categories are confused, but the current extracted artifact is primarily a visualization; the notebook did not export the underlying matrix as a separate CSV. Accordingly, the manuscript does not assert unverified numeric confusion pairs. The printed classification report is the source for the class-wise numbers above.

E. Streamlit Inference Interface and Deployment Workflow

The trained CNN was integrated into an interactive Streamlit application to demonstrate the complete end-to-end inference pipeline. The user interface allows users to upload single leaf images in JPG, JPEG, PNG, or WEBP formats. Prior to passing the image to the network, the preprocessing pipeline applies EXIF orientation normalization, transforms the image to three-channel RGB, resizes the spatial dimensions to 224×224 pixels, and rescales raw pixel intensities to the interval $[0, 1]$ using division by 255, matching the input constraints enforced during training.

Fig. 8 illustrates a representative prediction within the application. The test input is assigned to the class *Tomato Yellow Leaf Curl Virus* with an output confidence score of 100.00%. It is critical to note that this percentage represents an isolated softmax activation value rather than an empirically calibrated posterior probability or an indicator of generalized classification accuracy.

The application renders the input image alongside the top predicted class name by resolving the predicted integer class index through a pre-saved class mapping JSON dictionary. Preserving exact index-to-label ordering between model serialization and inference runtime is mandatory to prevent silent misclassification. Furthermore, the interface includes heuristic threshold warnings when the highest softmax score falls below 85.00%, prompting alternative candidate classes; however, this rule serves purely as an interface aid rather than a validated out-of-distribution rejection mechanism.

Consequently, the deployment interface functions strictly as an educational prototype and technical feasibility demonstration rather than a production-ready agronomic diagnostic tool. Unconstrained field imagery introduces domain shift factors such

as non-uniform natural illumination, occluded backgrounds, multi-leaf overlaps, and variable symptom manifestations that are absent from controlled laboratory datasets like PlantVillage. Without extensive prospective testing on real-world field distributions and rigorous confidence calibration (e.g., temperature scaling or expected calibration error evaluation), softmax outputs should not be interpreted as authoritative agronomic guidance.

VIII. Discussion

A. Interpretation of the measured performance

An 89.12% validation accuracy is a useful baseline for the implemented model, but its meaning is bounded by the protocol. It was measured on the same validation subset used to select the best checkpoint. It should not be compared numerically with the 99.35% held-out test accuracy in Mohanty *et al.* as though the two models were evaluated in the same experiment [2]. The older study's external-image result is a warning that controlled-dataset scores do not guarantee field performance.

The macro F1 of 0.8762 and weighted F1 of 0.8904 indicate that aggregate performance is not uniform across classes. The class-wise report gives a more actionable view: tomato early blight and potato healthy have low recall. The class count intervention does not eliminate imbalance, and the current paper cannot establish that it improved performance because no otherwise-identical model was trained without that intervention. A causal claim such as "balancing improved accuracy" would therefore be unsupported.

B. Class imbalance and data curation

Capping three high-count classes can reduce the influence of those classes, but random deletion discards data and leaves other class counts untouched. It can also change which visual examples the model sees. More principled alternatives include class-weighted loss, balanced mini-batch sampling, focal loss, or controlled over/undersampling, but each requires an experiment. Any comparison should keep the train-validation protocol, random seed, model, and training budget fixed and repeat across multiple seeds where feasible.

The dataset reduction should be redesigned as a non-destructive derived dataset with a manifest. It should also be evaluated by publishing per-class counts before and after reduction. The observed mismatch between the initial count of 54,305 and generator total of 44,351 cannot be fully audited from the current outputs alone. The notebook shows the three class reductions, but lacks a comprehensive file-level ledger to explain all differences. This is a reproducibility limitation.

C. Overfitting and architecture size

The dense classifier contributes 47.776 million of the model's 47.805 million parameters. Such a large dense head increases the number of trainable parameters relative to the convolutional feature extractor. Training accuracy of 99.74% alongside 89.12% validation accuracy suggests that the model fits the training set much more closely than it generalizes to the validation set. The validation loss increase in epochs 4–5 also indicates instability across the short training trajectory.

Potential mitigations include dropout, weight decay, data augmentation, early stopping, or replacing Flatten with global average pooling. These are recommendations for future experiments, not results of the present study. A smaller head may improve memory efficiency and generalization, but it could also change representational capacity; empirical comparison is needed.

D. Dataset shift and practical deployment

PlantVillage's value as a controlled benchmark is not the same as field realism. Images captured outside the dataset can include complex backgrounds, multiple leaves, scale variations, and camera artifacts. Mohanty *et al.*'s difference between held-out and external-image accuracy demonstrates the risk directly [2]. The present project has no field dataset, no out-of-distribution evaluation, and no expert-verified deployment trial. Therefore, the Streamlit app should be framed as an educational prototype or decision-support demonstration, not a replacement for agronomist diagnosis.

The app's confidence display should be treated carefully. The threshold of 85% is an interface rule, not a validated safety threshold. The shown score of 100.00% should not be interpreted as certainty. Before deployment, confidence calibration and abstention behavior should be tested, including images that do not contain a leaf or belong to an unseen disease class.

E. Reproducibility and reporting

The notebook provides valuable evidence: model definition, training logs, class-wise report, confusion matrix, saved label mapping, and plots. However, the exact dataset copy and image deletion manifest are absent from the uploaded artifacts, package versions are not pinned, and the final model binary is not among the files audited here. The notebook records that the model was saved to Drive, but this paper has not independently loaded the binary artifact. The report is therefore reproducible at the configuration-and-output level but not fully executable from the supplied files alone.

The app source and label mapping are available. Since the mapping must match the output index order, a robust release should version the model and class mapping together, record SHA-256 hashes, pin the runtime dependencies, and run a smoke test on known examples. The notebook's inference helper and app preprocessing should be tested for parity, especially RGB conversion and image orientation.

IX. Threats to Validity

A. Internal validity

The same validation set is used for checkpoint selection and final reported evaluation. This can make the score optimistic compared with a fully held-out test set. There is no comparison against an unbalanced baseline, class-weighted model, pretrained network, or alternative architecture, so the effect of the class-reduction intervention and architecture choices is unknown.

B. Construct validity

Accuracy and F1 describe label agreement on the chosen dataset; they do not measure agronomic usefulness, disease severity estimation, or downstream yield outcomes. The class labels are treated as mutually exclusive categories, which may not represent cases with multiple simultaneous symptoms.

C. External validity

The images are from a curated PlantVillage color directory. There is no evaluation on field photographs, different acquisition devices, or unseen environments. Performance claims should not be generalized beyond this dataset split.

D. Reproducibility validity

The notebook output lacks a removed-file manifest, exact GPU model, pinned package versions, and a separate test artifact. The saved model binary is referenced in notebook output but was not part of the uploaded files examined for this manuscript. These omissions limit independent rerunning and exact replication.

X. Ethical and Practical Considerations

A wrong disease label can lead to unnecessary pesticide use, delayed treatment, or crop loss if users treat the prediction as authoritative. The interface should communicate uncertainty, encourage expert confirmation, and avoid recommending chemical treatment unless such recommendations are supported by validated agronomic guidance. Dataset and model limitations should be visible to users. The model should not be marketed as field-validated until prospective evaluation has been performed.

XI. Future Work

A rigorous next phase should prioritize:

1) **Independent evaluation:** preserve an untouched test set, ideally using leaf-group-aware splits where metadata permits, and report all final metrics only after model selection is complete.
2) **Class-count audit:** save complete per-class counts before and after curation and record the exact removed file list.
3) **Controlled imbalance experiment:** compare the current capping approach with an unmodified baseline, class-weighted cross-entropy, balanced sampling, and focal loss under the same split and training budget.
4) **Architecture study:** compare the current large dense head with global average pooling and at least one pretrained backbone; report parameter count, inference latency, memory, and validation/test metrics.
5) **Repeated trials:** train each candidate across multiple seeds and report mean and standard deviation to estimate variability.
6) **Calibration and abstention:** evaluate reliability diagrams, Brier score, expected calibration error, and a reject option for low-confidence or unknown inputs.
7) **External validation:** collect independently labelled field images across lighting, backgrounds, growth stages, and devices, with agronomist review and a clear data-use protocol.
8) **Deployment hardening:** pin dependencies, test model/label-map consistency, add image-format tests, and document limitations in the app.

These items are proposals and should not be interpreted as completed experiments.

XII. Conclusion

This paper presents an evidence-grounded audit of a custom CNN for 38-class PlantVillage leaf-image classification. The executed notebook reports 89.12% validation accuracy, 0.4948 validation loss, macro F1 of 0.8762, and weighted F1 of 0.8904 after six epochs. The model contains 47.8 million trainable parameters, mostly in its dense classifier, and exhibits a substantial training-validation gap. Class-wise recall varies materially, with weak recall for Tomato Early blight and Potato healthy despite strong results for several other classes.

The targeted class-count reduction caps three classes at 2,000 images each but does not balance all classes, and no controlled experiment isolates its effect. Most importantly, no independent test set or field evaluation is available. The paper therefore supports a baseline result on the reported validation split, not a claim of state-of-the-art performance or real-world diagnostic reliability. The next scientifically meaningful step is not to claim a higher score, but to improve evaluation independence, class-wise auditability, and field generalization.

Acknowledgment

The authors acknowledge the PlantVillage dataset contributors and the open-source TensorFlow/Keras and Streamlit ecosystems used in the implementation. Any institutional acknowledgments should be added only if appropriate and approved.

Data and Code Availability

The dataset analyzed in this study is publicly accessible via the Kaggle PlantVillage repository at https://www.kaggle.com/datasets/abdallahalidev/plantvillage-dataset. The experimental Jupyter notebook, Streamlit application source code, class-index mapping dictionary, runtime dependency specifications, and extracted evaluation figures were utilized for this manuscript preparation. The trained model binary and intermediate runtime checkpoints were recorded to Google Drive and are available from the corresponding authors upon reasonable request. Redistribution and usage of dataset images are governed by the original open-access dataset terms and licensing.

References

[1] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics through machine learning and crowdsourcing," *arXiv preprint arXiv:1511.08060*, 2015. [Online]. Available: https://arxiv.org/abs/1511.08060

[2] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.

[3] K. P. Ferentinos, "Deep learning models for plant disease detection and diagnosis," *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018.

[4] E. C. Too, L. Yujian, S. Njuki, and L. Yingchun, "A comparative study of fine-tuning deep learning models for plant disease identification," *Computers and Electronics in Agriculture*, vol. 161, pp. 272–279, 2019.

[5] L. Pacal, I. Kunduracioglu, M. H. Alma, M. Deveci, S. Kadry, J. Nedoma, V. Slany, and R. Martinek, "A systematic review of deep learning techniques for plant diseases," *Artificial Intelligence Review*, vol. 57, no. 11, p. 304, 2024.

[6] TensorFlow Datasets, "PlantVillage dataset catalog," 2026, accessed Oct. 27, 2026. [Online]. Available: https://www.tensorflow.org/datasets/catalog/plant_village

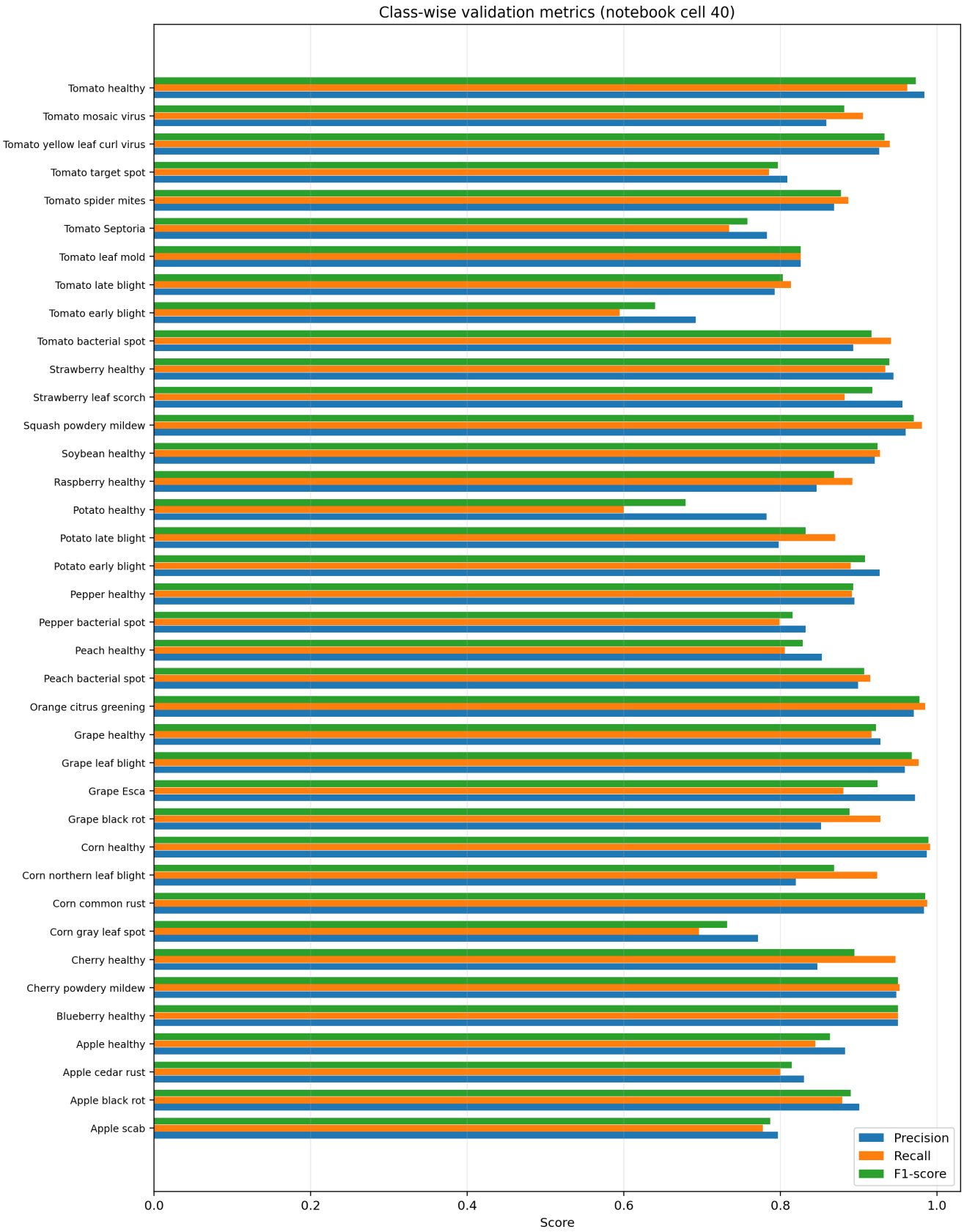


Fig. 6. Per-class precision, recall, and F1-score calculated directly from the classification report printed in notebook cell 40. Supports vary by class.

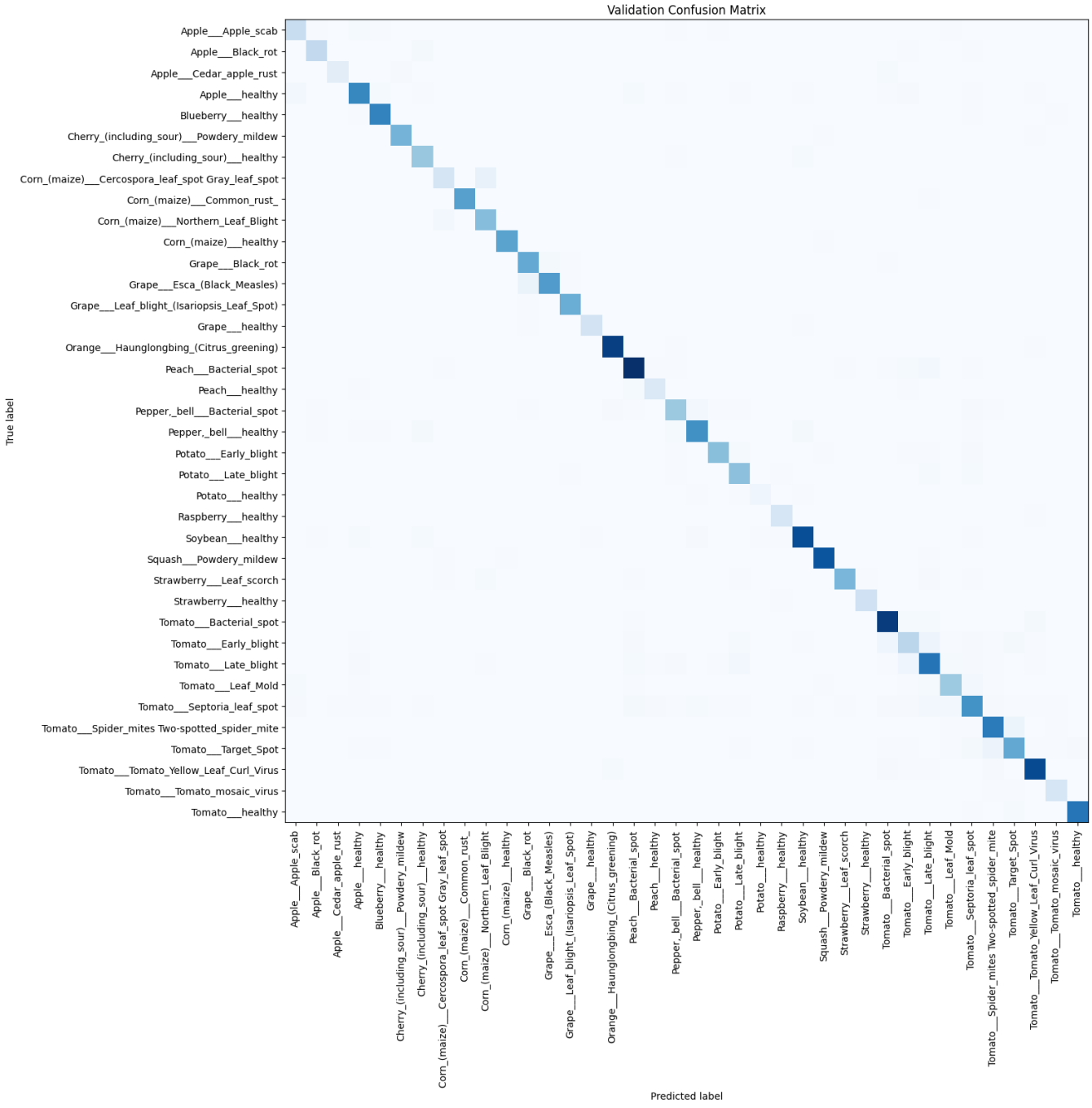


Fig. 7. Confusion matrix generated from validation predictions in notebook cell 41. The notebook's evaluation split is used; this is not an independent test-set matrix.

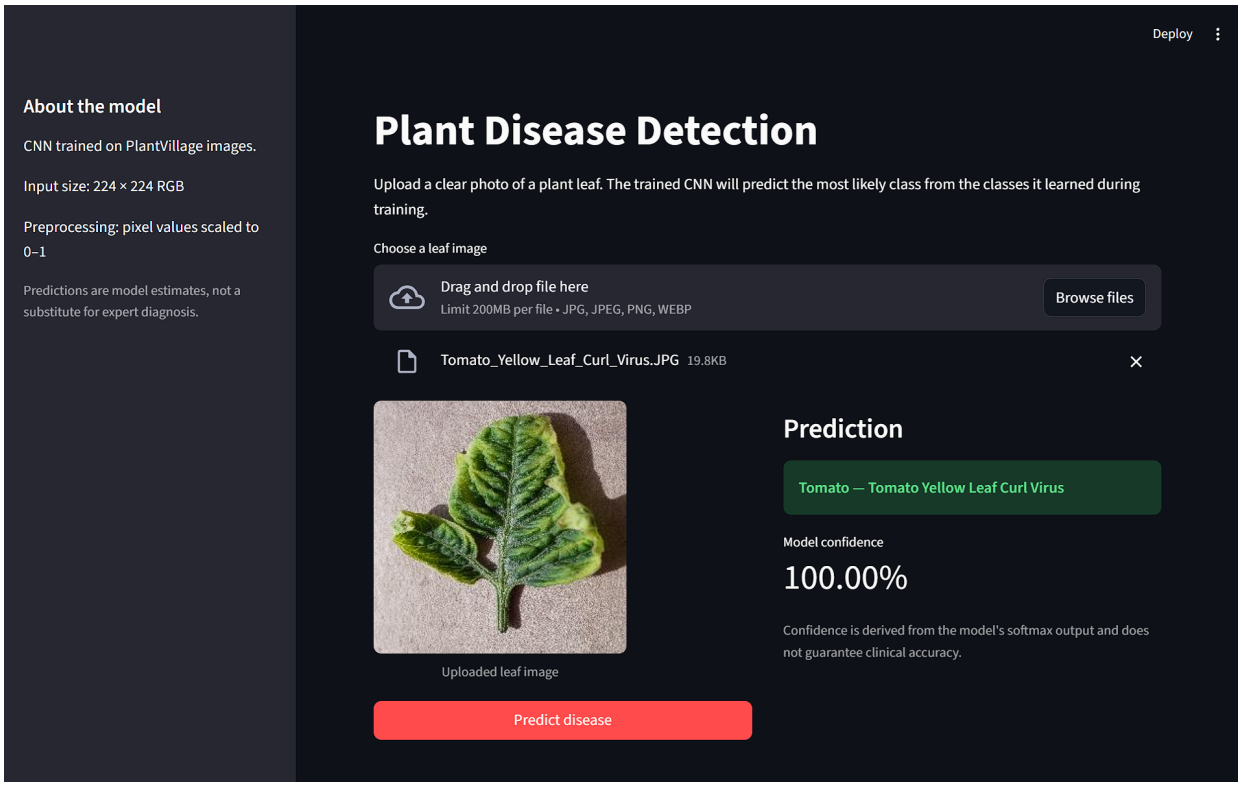


Fig. 8. Streamlit inference user interface demonstrating model prediction on a sample leaf image. The displayed 100.00% confidence is an uncalibrated maximum softmax score for an individual input and does not reflect holistic diagnostic accuracy or operational field reliability.